

```
import numpy as np
```

```
from scipy.stats import norm
from scipy.optimize import brentq
```

Core Functions:

d1 d 1, d 2 d 2, and Pricing

```
S = 100.0      # Spot price
K = 100.0      # Strike price
T = 1.0        # Time to expiry in years
r = 0.05       # Risk-free rate
q = 0.0        # Dividend yield
sigma = 0.20   # Volatility
option_type = "call"
```

```
def d1(S, K, T, r, q, sigma):
    return (np.log(S / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
```

```
def d2(S, K, T, r, q, sigma):
    return d1(S, K, T, r, q, sigma) - sigma * np.sqrt(T)
```

```
def bs_price(S, K, T, r, q, sigma, option_type):

    d1_val = d1(S, K, T, r, q, sigma)
    d2_val = d2(S, K, T, r, q, sigma)

    if option_type == "call":
        price = S * np.exp(-q * T) * norm.cdf(d1_val) - K * np.exp(-r * T) * norm.cdf(d2_val)
    elif option_type == "put":
        price = K * np.exp(-r * T) * norm.cdf(-d2_val) - S * np.exp(-q * T) * norm.cdf(-d1_val)
    else:
        raise ValueError(f"option_type must be 'call' or 'put', got '{option_type}'")

    return price
```

2 Price the Option

```
sigma = 0.5
d1_val = d1(S, K, T, r, q, sigma)
d2_val = d2(S, K, T, r, q, sigma)
Nd1_val = norm.cdf(d1_val)
Nd2_val = norm.cdf(d2_val)
price = bs_price(S, K, T, r, q, sigma, option_type)
```

```
print(f"d1      = {d1_val:.6f}")
print(f"d2      = {d2_val:.6f}")
print(f"Nd1      = {Nd1_val:.6f}")
print(f"Nd2      = {Nd2_val:.6f}")
print(f"\n price = ${price:.4f}")
```

```
d1      = 0.350000
d2      = -0.150000
Nd1      = 0.636831
Nd2      = 0.440382
```

```
price = $21.7926
```

Arbitrage Boundary Values

```
def price_bounds(S, K, T, r, q, option_type):
    pv_S = S * np.exp(-q * T) # PV of the stock (adjusted for dividends)
    pv_K = K * np.exp(-r * T) # PV of the strike
```

```

    if option_type == "call":
        lower = max(pv_S - pv_K, 0.0)
        upper = pv_S
    elif option_type == "put":
        lower = max(pv_K - pv_S, 0.0)
        upper = pv_K
    else:
        raise ValueError(f"option_type must be 'call' or 'put', got '{option_type}'")

    return lower, upper

lower, upper = price_bounds(S, K, T, r, q, option_type)

print(f"Lower bound : ${lower:.4f}")
print(f"BS price    : ${price:.4f}")
print(f"Upper bound : ${upper:.4f}")
print(f"\nBounds satisfied: {lower <= price <= upper}")

```

```

Lower bound : $4.8771
BS price    : $21.7926
Upper bound : $100.0000

Bounds satisfied: True

```

4. Implied Volatility

```

def implied_volatility(S, K, T, r, q, market_price, option_type,
                      sigma_lo=1e-6, sigma_hi=10.0, tol=1e-12):

    lower, upper = price_bounds(S, K, T, r, q, option_type)
    if market_price < lower:
        raise ValueError(f"Market price ${market_price:.4f} < lower bound ${lower:.4f}")
    if market_price > upper:
        raise ValueError(f"Market price ${market_price:.4f} > upper bound ${upper:.4f}")

    def objective(sigma):
        return bs_price(S, K, T, r, q, sigma, option_type) - market_price

    sigma_iv = brentq(objective, sigma_lo, sigma_hi, xtol=tol)
    return sigma_iv

```

```

market_price = 8
implied_volatility(S, K, T, r, q, market_price, option_type,
                  sigma_lo=1e-6, sigma_hi=10.0, tol=1e-12)

```

```
0.13377582588953615
```

Start coding or generate with AI.

